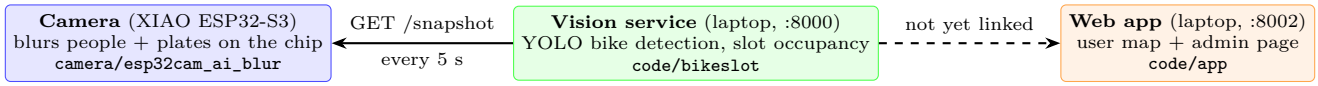
Find my Bike Slot – running the project on Windows

BaselHack 2026, team NextGen · repository `https://github.com/manuel-buser/NextGen_BaselHack2026` · all commands for PowerShell



The camera only answers requests; it never sends anything on its own (pull mode). All devices must be on the same 2.4 GHz Wi-Fi (phone hotspot or venue Wi-Fi).

1. Install once

Tool	Notes
Git	`https://git-scm.com`
Python 3.12	`https://python.org`; tick "Add to PATH". Check: `py -3.12 -version`
VS Code + **PlatformIO IDE**	Extensions → "PlatformIO IDE". Brings the ESP32 toolchain itself (into `%USERPROFILE%\.platformio`, outside the repo).
uv (only for the web app)	`py -m pip install uv`

2. Get the code

```
cd $env:USERPROFILE\Desktop
git clone https://github.com/manuel-buser/NextGen_BaselHack2026.git
cd NextGen_BaselHack2026
git pull                                  # later: fetch the team's newest changes
```

3. Camera firmware

Settings (local file, never committed – it holds Wi-Fi passwords):

```
cd camera\esp32cam_ai_blur
copy src\config.example.h src\config.h
notepad src\config.h        # WIFI_NETWORKS: {"name", "password"}, tried in this order
```

Build, flash, check (camera on USB-C; the first build takes 20–30 min):

```
$pio = "$env:USERPROFILE\.platformio\penv\Scripts\pio.exe"
& $pio run -j 2                          # first time: run twice (see note)
& $pio run -j 2 -t upload --upload-port COM8
& $pio device monitor                    # shows "Wi-Fi connected to ..., IP x.x.x.x"; quit: Ctrl+C
```

Note: the first build stops with "model not downloaded yet – build again"; just run it again. Use your own COM port (VS Code: PIO Home → Devices; Bluetooth ports are not it). `-j 2` avoids running out of memory.

Camera endpoints (browser on the same Wi-Fi):

`http://<camera-ip>/`	live view and camera settings (auto-exposure switch included)
`http://<camera-ip>/snapshot`	one JPEG, people and plates already pixelated – this is what the service fetches
`http://<camera-ip>/status`	settings and auto-tuning state as JSON

4. Vision service (bike detection)

From the repository root, in a virtual environment (the `.venv` folder is git-ignored):

```
py -3.12 -m venv .venv
.venv\Scripts\python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
.venv\Scripts\python -m pip install -r requirements.txt
```

Note: with an NVIDIA GPU, install the CUDA build of torch instead (see `requirements.txt`). The YOLO model downloads itself on the first start.

Start it, pointing it at the camera's IP from the serial monitor:

```
$env:BIKESLOT_PULL_CAMERAS = "live01=http://<camera-ip>/snapshot"
$env:BIKESLOT_DEVICE = "cpu"             # leave out if you have an NVIDIA GPU
.venv\Scripts\python -m uvicorn --app-dir code bikeslot.api:app --host 0.0.0.0 --port 8000
```

Open `http://localhost:8000/` (debug view) and `http://localhost:8000/docs` (API). **Without a camera**, feed it the sample pictures from a second terminal:

```
.venv\Scripts\python code\scripts\replay.py --camera cam01 --source assets\pictures\GoldenDataSet --interval 3 --
    loop
```

Note: a phone on the same Wi-Fi reaches the service at `http://<laptop-ip>:8000/` once Windows Firewall allows Python on port 8000 (Windows asks on the first start: allow "Private networks").

5. Web app (user map and admin page)

```
cd code\app
uv sync
uv run uvicorn bikeslot.main:app --port 8002
```

Open `http://localhost:8002` (user page) and `http://localhost:8002/admin`. Racks are configured in `racks.yaml`. For a demo with made-up racks filling up during a simulated day:

```
$env:BIKESLOT_RACKS = "racks.demo.yaml"; uv run uvicorn bikeslot.main:app --port 8002
uv run python tools\demo_data.py --app http://localhost:8002     # second terminal
```

Note: the app talks to a detector through `DETECTOR_URL` (`POST /analyze`); the vision service of section 4 does not offer that interface yet, so the app runs with its built-in stand-in detector (`DETECTOR_URL=fake`, the default). Details: `code/app/README.md`, `documentation/CONTRACTS.md`.

6. Working with git

```
git pull                                  # before you start
git status                                # what changed
git add <files>
git commit -m "short description"
git push
```

Never commit (already in `.gitignore`): `config.h` (Wi-Fi passwords), `.venv/`, `.pio/`, `managed_components/`, `*.pt` model files, photos showing people or licence plates. Check before a push: `git status` must not list any of them.

7. When something goes wrong

Message	Fix
`Could not open COM8 ...` `Access is denied`	A serial monitor still holds the port. Stop it (Ctrl+C in its terminal) or: `Get-CimInstance Win32_Process | ? CommandLine -match 'device monitor' | % { Stop-Process -Id $_.ProcessId -Force }`
Upload finds no board	Hold **BOOT** on the XIAO while plugging in, then upload again. Use a data cable.
`model not downloaded yet`	Normal on the first build: build again.
Build errors about `WIFI_NETWORKS`	`config.h` has an old format: compare with `config.example.h` (two values per network).
Camera page stays on "loading..."	Laptop and camera on different networks, or a slow venue Wi-Fi: use the phone hotspot for both, reload with Ctrl+F5.
Camera joins the wrong network	It tries `WIFI_NETWORKS` in order and moves on after 4 failures: put the wanted network first and upload again.
VS Code asks "Select a Kit"	CMake Tools extension, not needed: press Esc, use PlatformIO's Build (tick icon). Disable CMake Tools for this workspace.